

Trabajo Práctico 2 - Grupo 02

Reseñas de Amazon en español

Integrantes:

- Bermudez, Agustin. Padrón: 111863
Email: abermudez@fi.uba.ar
- Calderón, Tiago. Padrón: 111894
Email: tcalderon@fi.uba.ar
- Gonzalez Pautaso, Mateo. Padrón: 111699
Email: magonzalezp@fi.uba.ar
- Moreyra, Santiago. Padrón: 110531
Email: smoreyra@fi.uba.ar
- Nieves, Maylen. Padrón: 111271
Email: mnieves@fi.uba.ar

Introducción.....	5
Breve descripción del problema a resolver.....	5
Breve descripción del dataset.....	5
Breve comentario de las técnicas exploradas, pruebas realizadas y modificaciones sobre el dataset.....	5
Técnicas de preprocesamiento de datos utilizadas.....	6
Hipótesis o supuestos tomados.....	7
Cuadro de Resultados.....	8
Descripción de Modelos.....	9
Bayes Naive.....	9
Introducción.....	9
Descripción de versiones.....	9
v1: MultinomialNB + TF-IDF, priors por defecto (F1 Kaggle: 0.57797).....	9
v2: MultinomialNB + TF-IDF, priors uniformes (F1 Kaggle: 0.65808).....	9
v3: MultinomialNB + TF-IDF con GridSearch (F1 Kaggle: 0.66059).....	9
v4: MultinomialNB + BoW, priors uniformes (F1 Kaggle: 0.66189).....	9
v5: MultinomialNB + BoW con GridSearch (F1 Kaggle: 0.66461).....	10
v6: GaussianNB + FastText (F1 Kaggle: 0.5072).....	10
Modelo ganador: v5 - MultinomialNB + BoW + GridSearch.....	10
Preprocesamiento.....	10
Representación del texto.....	10
Clasificador.....	10
Búsqueda de hiperparámetros.....	10
Mejores hiperparámetros encontrados.....	11
Entrenamiento final.....	11
Resultados.....	11
Técnicas adicionales exploradas.....	11
Random Forest.....	12
Introducción.....	12
Descripción de versiones.....	12
v1: RF + TF-IDF, baseline (F1 Kaggle: 0.60657).....	12
v2: RF + TF-IDF + RandomizedSearch 20 iter (F1 Kaggle: 0.65151).....	12
v3: RF + BoW + RandomizedSearch 20 iter (F1 Kaggle: 0.65088).....	12
v4: RF + TF-IDF + RandomizedSearch 40 iter (F1 Kaggle: 0.65170).....	12
v5: RF + TF-IDF + SelectKBest + búsqueda en dos fases (F1 Kaggle: 0.64959).....	12
v6: RF + TF-IDF + meta-features + dataset LLM balanceado (F1 Kaggle: 0.61096).....	12
Modelo ganador: v4 - RF + TF-IDF + RandomizedSearch 40 iteraciones.....	13
Preprocesamiento.....	13
Representación del texto.....	13
Clasificador.....	13
Búsqueda de hiperparámetros.....	13
Mejores hiperparámetros encontrados.....	13

Entrenamiento final.....	14
Resultados.....	14
Técnicas adicionales exploradas.....	14
XGBoost.....	15
Introducción.....	15
Descripción de versiones.....	15
v1: XGBoost + TF-IDF, sin búsqueda de hiperparámetros (F1 Kaggle: 0.65274).....	15
v2: XGBoost + BoW, sin búsqueda de hiperparámetros (F1 Kaggle: 0.66021)....	15
v3: XGBoost + TF-IDF + RandomizedSearch 20 iter (F1 Kaggle: 0.66894).....	15
v4: XGBoost + BoW + RandomizedSearch 20 iter (F1 Kaggle: 0.67682).....	15
v5: XGBoost + BoW + RandomizedSearch 40 iter, grilla refinada (F1 Kaggle: 0.67792).....	15
v6: XGBoost + BoW/TF-IDF + dataset aumentado (F1 Kaggle: 0.68157).....	16
v7: XGBoost + TF-IDF/BoW + meta-features, sin búsqueda (F1 Kaggle: 0.66826).	16
v8: XGBoost + TF-IDF/BoW + meta-features + RandomizedSearch (F1 Kaggle: 0.66824).....	16
Modelo ganador: v6 - XGBoost + BoW + dataset aumentado.....	16
Motivación y dataset.....	16
Preprocesamiento.....	16
Búsqueda de hiperparámetros.....	16
Manejo del desbalance.....	17
Resultados en validación (v6.3 - BoW, dataset aumentado EDA).....	17
Técnicas adicionales exploradas.....	18
Red Neuronal.....	19
Introducción.....	19
Descripción de versiones.....	19
v1: BETO + dataset original, configuración estándar (F1 Kaggle: 0.72048).....	19
v2: BETO + dataset EDA 67k balanceado (F1 Kaggle:0.70313).....	19
v3: BETO + dataset LLM aumentado (~66.5k) (F1 Kaggle: 0.72463).....	19
v4: BETO + dataset original, lr=1e-5, 5 épocas (F1 Kaggle: 0.72162).....	19
v5: BETO + dataset original, max_length=192 (F1 Kaggle: 0.72132).....	20
v6: BETO + dataset EDA 67k, lr=3e-5, 2 épocas (F1 Kaggle: 0.70365).....	20
v7: BETO + dataset LLM, lr=1e-5, 5 épocas (F1 Kaggle: 0.72140).....	20
v8: XLM-RoBERTa-Large + dataset original (F1 Kaggle: 0.72765).....	20
v9: BETO + dataset original, lr=1e-5, 5 épocas, post-RoBERTa (F1 Kaggle: 0.72596).....	20
v10: XLM-RoBERTa-Base + dataset LLM (F1 Kaggle: 0.71582).....	20
v11: FastText + BiLSTM + MultiHead Attention (F1 Kaggle: 0.66698).....	20
v12: XLM-RoBERTa-Base + split limpio + EDA aug + class weights (F1 Kaggle: 0.72371).....	21
v13: XLM-RoBERTa-Large + split limpio + EDA aug + class weights (F1 Kaggle: 0.73367).....	21
Arquitectura del modelo y justificación.....	21

Modelo ganador: v13 - XLM-RoBERTa-Large + split limpio + EDA aug + class weights.....	21
Modelo base y motivación.....	21
Preprocesamiento.....	22
Dataset y estrategia de split limpio.....	22
Hiperparámetros.....	22
Manejo del desbalance.....	22
Entrenamiento y selección del mejor modelo.....	23
Resultados.....	23
Técnicas adicionales exploradas.....	23
Arquitectura.....	23
Entrenamiento en 2 fases.....	23
Ensamble.....	25
Introducción.....	25
Descripción de versiones.....	25
v1: XLM-RoBERTa-base + XgBoost + Naive Bayes (F1 Kaggle: 0.68914).....	25
v2: BETO v3 + BETO v9 + RoBERTa-BNE n1 (F1 Kaggle: 0.73555).....	25
v3: RoBERTa-BNE n1 + XGBoost v6+ Random Forest v4 (F1 Kaggle: 0.73930).....	25
v4: RoBERTa-BNE n1 + XgBoost + Naive Bayes con dataset EDA (F1 Kaggle: 0.73345).....	26
v5: RoBERTa-BNE n1 + XGBoost + NB con dataset LLM (F1 Kaggle: 0.73345).....	26
v6: XLM-RoBERTa-base LLM + RF v5 + XGBoost (F1 Kaggle: 0.72157).....	26
v7: BETO v3.1 + XGBoost v6 (F1 Kaggle: 0.72828).....	26
v8: XLM-RoBERTa N13 Large + RoBERTa-BNE n1 + XGBoost + RF + Naive Bayes (F1 Kaggle: 0.72695).....	26
v9: XLM-RoBERTa-Large N13 + XGBoost + RF + Complement NB + Logistic Regression (F1 Kaggle: 0.73746).....	27
Modelo ganador: v3 - RoBERTa-BNE n1 + XGBoost v6 + RF v4.....	27
Motivación.....	27
Modelos ensamblados.....	27
Pesos.....	27
Preprocesamiento.....	27
Mecanismo de ensamble.....	28
Resultados.....	28
Técnicas adicionales exploradas.....	28
Conclusiones generales.....	29
Tiempo dedicado.....	31

Introducción

Breve descripción del problema a resolver

Este informe documenta el trabajo realizado en el marco de la competencia de Kaggle propuesta por la cátedra. El objetivo es clasificar reseñas de productos de Amazon en español según el sentimiento expresado por el usuario, en tres categorías: negativo, neutro o positivo. Se trata de un problema de clasificación multiclase evaluado mediante la métrica F1-macro, que pondera por igual el desempeño en las tres clases independientemente de su frecuencia en el dataset. Para abordarlo se entrenaron y compararon distintos clasificadores (Naive Bayes, Random Forest, XGBoost, BETO y un ensamble de al menos 3 de los modelos entrenados) combinados con cuatro representaciones de texto (Bag of Words, TF-IDF y embeddings densos, tanto estáticos vía FastText como contextuales vía BETO).

Breve descripción del dataset

El conjunto de entrenamiento contiene 51.000 registros y 3 columnas (id, text, label), sin valores nulos y con 238 duplicados exactos por texto. El conjunto de test contiene 8.500 registros sin la columna label, con 20 duplicados exactos. Las clases están moderadamente desbalanceadas: negativa y positiva representan ~40% cada una, mientras que neutra representa solo ~20%.

Una particularidad encontrada durante el análisis exploratorio es la ambigüedad léxica de algunas reseñas. Por ejemplo, en una frase como "está re bueno mal", donde el segundo adverbio intensifica al primero en lugar de negarlo, un modelo basado en frecuencia de palabras (Bag of Words) podría asociar erróneamente el término "mal" a un sentimiento negativo, cuando en realidad la reseña es positiva. Este tipo de casos motivó decisiones de preprocesamiento orientadas a preservar negaciones e intensificadores como señales centrales del sentimiento (más detallado en la sección de preprocesamiento).

Breve comentario de las técnicas exploradas, pruebas realizadas y modificaciones sobre el dataset

Se implementaron dos pipelines de limpieza de texto diferenciados según el tipo de modelo consumidor: uno minimalista para BETO y embeddings contextuales, cuyo tokenizer espera texto natural (con mayúsculas y puntuación originales), y otro más agresivo para los modelos clásicos (BoW/TF-IDF/embeddings estáticos), que se benefician de reducir el espacio de features mediante lematización y eliminación de stopwords.

Se exploraron cuatro representaciones del texto: Bag of Words, TF-IDF, embeddings estáticos preentrenados con FastText (cc.es.300.bin, 300 dimensiones) y la tokenización subword propia de los modelos transformer.

Adicionalmente, se intervino el dataset de entrenamiento en dos frentes:

- **Balanceo de clases:** dado que la clase neutra está subrepresentada (~20% frente a ~40% de las otras dos), se generaron ejemplos sintéticos adicionales mediante dos estrategias de data augmentation: EDA (Easy Data Augmentation, reemplazo de sinónimos vía WordNet, inserción, swap y borrado aleatorio de palabras) y generación con un LLM (Llama 3.1 8B vía API de Groq), produciendo dos variantes balanceadas del dataset de entrenamiento.

- **Meta-features:** se extrajeron 12 features numéricas adicionales a partir del texto crudo (longitud en tokens y caracteres, conteo de signos de exclamación/interrogación, proporción de mayúsculas, presencia de signos repetidos, cantidad de negaciones y puntajes de polaridad promedio según SentiWordNet). Para integrarlas al pipeline de los modelos clásicos se desarrolló la clase `VectorizerPlusFeaturesPipeline`, que combina vectorizador (BoW/TF-IDF) + meta-features + clasificador en un único objeto compatible con la interfaz `fit/predict` de scikit-learn.

Por último, se detallan las implementaciones propias desarrolladas por el equipo, reutilizadas de forma consistente en todas las notebooks del trabajo:

- `preprocessing.py`: funciones de limpieza de texto (`clean_minimal`, `clean_classical`).
- `data_utils.py`: split estratificado reproducible (`get_split`), factorías de vectorizadores (`make_bow`, `make_tfidf`), extracción de meta-features (`extract_features`) y `VectorizerPlusFeaturesPipeline`.
- `embeddings.py`: `FastTextVectorizer`, wrapper sklearn-compatible sobre el modelo preentrenado de `FastText`.
- `evaluation.py`: función `evaluate()` que reporta F1-macro, precision, recall, accuracy, classification report y matriz de confusión.
- `io_utils.py`: persistencia de modelos (`save_model`, `load_model`) y generación de archivos de submission para Kaggle (`make_submission`).
- `augment_data.py`: generación de ejemplos sintéticos mediante EDA con WordNet en español.
- `augment_data_llm.py`: generación de ejemplos sintéticos mediante un LLM (Groq API).

Técnicas de preprocesamiento de datos utilizadas

El análisis exploratorio mostró que el texto es naturalmente limpio: no se detectó HTML, URLs, menciones, hashtags ni direcciones de email, y la presencia de emojis es marginal (0.29%). Esto permitió optar por una limpieza minimalista en lugar de un pipeline de normalización agresivo, evitando descartar información potencialmente relevante.

Se definieron dos funciones de limpieza con propósitos distintos:

- `clean_minimal` (para BETO/embeddings contextuales): normalización Unicode NFKC y colapso de espacios en blanco, sin modificar mayúsculas ni puntuación, ya que el tokenizer de BETO es cased y distingue, por ejemplo, "BUENO" de "bueno".

- `clean_classical` (para BoW/TF-IDF/embeddings estáticos): conversión a minúsculas, lematización con spaCy (`es_core_news_sm`, ej. "corriendo" → "correr"), eliminación de stopwords clásicas del español y descarte de tokens de menos de 2 caracteres. Del conjunto de stopwords se excluyeron deliberadamente las negaciones ("no", "ni", "nunca", "tampoco", "nada", etc.), intensificadores/atenuadores ("muy", "poco", "demasiado", "bastante", etc.) y adversativas ("pero", "sino", "aunque"), por ser determinantes para el sentimiento expresado. Esta limpieza redujo en un 39.4% la cantidad total de tokens del corpus (mediana de 22 a 13 tokens por reseña), sin generar reseñas vacías.

Para la vectorización (BoW y TF-IDF) se utilizaron n-gramas de orden 1 a 2 (unigramas + bigramas, para capturar construcciones como "no funciona" o "muy bueno"), `min_df=5` y `max_df=0.95` para filtrar tanto términos demasiado raros (typos, nombres propios) como demasiado frecuentes (poco discriminativos), y un patrón de tokenización que admite guiones internos (ej. "anti-robo") y excluye palabras de un solo carácter.

Hipótesis o supuestos tomados

A partir de los hallazgos del análisis exploratorio, se adoptó una serie de supuestos que guiaron tanto las decisiones de preprocesamiento como las de modelado:

- Al evaluarse con F1-macro, el desbalance de clases (40/20/40) es crítico aunque moderado: un modelo trivial que prediga siempre la clase mayoritaria obtiene buen accuracy pero mal F1-macro, por lo que se decidió evaluar todos los modelos con y sin técnicas de rebalanceo.
- Las negaciones e intensificadores son determinantes del sentimiento y deben preservarse explícitamente en cualquier pipeline de limpieza que elimine stopwords, ya que su pérdida colapsa pares semánticamente opuestos (ej. "me gustó" vs. "no me gustó") al mismo vector de features.
- El uso de mayúsculas sostenidas (ALL CAPS, presente en ~4.1% de las reseñas) es una señal de intensidad emocional y se pierde al aplicar lowercase, por eso se mantiene el texto sin normalizar para los modelos basados en transformers.
- La clase neutra no representa ausencia de opinión sino sentimiento mixto (reseñas que valoran algo positivamente y critican otra cosa en la misma oración). El análisis de vocabulario discriminativo (χ^2 sobre TF-IDF) mostró que su vocabulario tiene scores sensiblemente más bajos que el de las clases negativa y positiva, y comparte términos con ambos extremos, lo cual permite anticipar que será la clase más difícil de clasificar para todos los modelos.
- Las reseñas negativas tienden a ser más extensas que las positivas, asumiendo que los usuarios se explayan más al quejarse que al elogiar un producto. Esto fue contemplado al definir `max_length=128` para BETO, valor que cubre el percentil 99 de longitud en tokens sin truncar información relevante en la inmensa mayoría de los casos.

Cuadro de Resultados

Modelo	F1-macro Test	Precision Test	Recall Test	Accuracy Test	Kaggle
Bayes Naive	0.6507	0.6521	0.6516	0.6921	0.66461
Random Forest	0.6418	0.6420	0.6416	0.6898	0.65170
XGBoost	0.7513	0.7516	0.7510	0.7512	0.68157
Red Neuronal	0.7571	0.7568	0.7616	0.7910	0.73367
Ensamble	0.7932	0.7957	0.7913	0.8326	0.73930

El modelo seleccionado como mejor predictor del trabajo práctico es el Ensamble v3 (RoBERTa-BNE n1 + XGBoost v6+ Random Forest v4), con un F1-macro de 0.73930 en el tablero público de Kaggle y 0.7932 en validación local. Esta elección se justifica por la diversidad de sus componentes: combina un transformer de gran capacidad que captura semántica contextual profunda con dos modelos clásicos de familias distintas (boosting y bagging), cuyas fortalezas se complementan y cuyos errores tienden a compensarse mutuamente mediante soft voting ponderado.

Descripción de Modelos

Bayes Naive

Introducción

Bayes Naive es el modelo canónico para clasificación de texto: rápido de entrenar, transparente (sus pesos son log-probabilidades por palabra y clase) y con un costo computacional muy bajo en entrenamiento y predicción. Se utilizó como modelo baseline: cualquier modelo más sofisticado debía superar su F1-macro, caso contrario indicaría un error en el pipeline. Se exploraron seis versiones progresivas, variando la representación del texto (Bag of Words y TF-IDF), los priors de clase y la búsqueda de hiperparámetros.

Descripción de versiones

v1: MultinomialNB + TF-IDF, priors por defecto (F1 Kaggle: 0.57797)

Baseline inicial. Pipeline con TF-IDF (configuración por defecto) y `MultinomialNB(alpha=1.0)`. Los priors se estimaron a partir de la frecuencia de cada clase en el conjunto de entrenamiento, lo que penalizó fuertemente a la clase neutra (subrepresentada, ~20% del dataset). En validación la clase neutra colapsó a $F1 \approx 0.15$, arrastrando el macro promedio.

v2: MultinomialNB + TF-IDF, priors uniformes (F1 Kaggle: 0.65808)

Se reemplazaron los priors estimados por priors uniformes (`class_prior=[1/3, 1/3, 1/3]`), forzando al modelo a no favorecer a priori ninguna clase. El único cambio respecto a v1 fue ese parámetro, todo lo demás permaneció igual. El F1-macro subió casi 7 puntos, prácticamente todo proveniente de la clase neutra, cuyo F1 pasó de 0.15 a 0.38.

v3: MultinomialNB + TF-IDF con GridSearch (F1 Kaggle: 0.66059)

Se incorporó `GridSearchCV` (5 folds, scoring F1-macro) para buscar el mejor valor del parámetro de suavizado de Laplace `alpha` y si conviene estimar los priors o forzarlos uniformes. La grilla explorada fue:

- `nb__alpha`: [0.05, 0.07, 0.1, 0.13, 0.15, 0.2, 0.3]
- `nb__fit_prior`: [True, False]

Los mejores hiperparámetros encontrados fueron `alpha=0.3` y `fit_prior=False` (priors uniformes), confirmando el hallazgo de v2. El `alpha` bajo reduce el suavizado, confiando más en las frecuencias observadas.

v4: MultinomialNB + BoW, priors uniformes (F1 Kaggle: 0.66189)

Se reemplazó la representación TF-IDF por Bag of Words (conteos crudos), manteniendo `alpha=1.0` y `class_prior=[1/3, 1/3, 1/3]`. BoW superó levemente a TF-IDF con los mismos priors, sugiriendo que la normalización por IDF no aportaba en este contexto.

v5: MultinomialNB + BoW con GridSearch (F1 Kaggle: 0.66461)

Combina la representación BoW (mejor que TF-IDF, según v4) con búsqueda exhaustiva de hiperparámetros. Descripción completa en la sección siguiente.

v6: GaussianNB + FastText (F1 Kaggle: 0.5072)

Se exploró una variante con embeddings densos de FastText (modelo pre entrenado `cc.es.300` de Meta, 300 dimensiones por documento como promedio de vectores de tokens). Como las representaciones de FastText son continuas, se usó `GaussianNB` en lugar de `MultinomialNB`, asumiendo distribución normal por clase y feature. Se buscó el mejor `var_smoothing` en escala logarítmica (`np.logspace(-12, -1, 12)`). A pesar de capturar semántica, el modelo obtuvo el peor resultado ($F1 \approx 0.50$), probablemente porque `GaussianNB` pierde la estructura sparse que favorece a `MultinomialNB` y porque el promedio de embeddings borra información discriminativa local.

Modelo ganador: v5 - MultinomialNB + BoW + GridSearch

Preprocesamiento

Se aplicó el pipeline `clean_classical` sobre los textos crudos, que incluye conversión a minúsculas, eliminación de puntuación y caracteres especiales, y remoción de stop words en español (usando spaCy con el modelo `es_core_news_sm`). El texto limpio se vectoriza con Bag of Words (conteos de n-gramas).

Representación del texto

Se usó `CountVectorizer` (BoW). Los hiperparámetros del vectorizador se incluyeron en la búsqueda:

- `bow__ngram_range`: unigramas (1,1) o unigramas+bigramas (1,2)
- `bow__max_features`: 30.000 o 50.000 tokens más frecuentes

Clasificador

`MultinomialNB`, cuyo único hiperparámetro relevante es `alpha` (suavizado de Laplace): evita probabilidades nulas para palabras no vistas en entrenamiento. Se buscó junto con si conviene estimar los priors o forzarlos uniformes:

- `nb__alpha`: [0.1, 0.5, 1.0, 2.0]
- `nb__fit_prior`: [True, False]

Búsqueda de hiperparámetros

`GridSearchCV` con validación cruzada de 5 folds, optimizando F1-macro, con `n_jobs=-1` para paralelización. La grilla completa tuvo $2 \times 2 \times 4 \times 2 = 32$ combinaciones.

Mejores hiperparámetros encontrados

Parámetro	Valor
<code>bow__ngram_range</code>	(1,2) - unigramas y bigramas
<code>bow__max_features</code>	30000
<code>nb__alpha</code>	2.0
<code>nb__fit_prior</code>	False (priors uniformes)

Entrenamiento final

El mejor estimador encontrado por `GridSearchCV` se reentrenó sobre la unión de train y validación completos antes de generar las predicciones de test, maximizando los datos disponibles para el modelo final.

Resultados

Métrica	Valor
F1-macro (validación)	0.6507
Precisión (validación)	0.6521
Recall (validación)	0.6516
Accuracy (validación)	0.6921
F1-macro (Kaggle)	0.66461

El modelo mostró buen desempeño en las clases negativa y positiva, con menor F1 en la clase neutra, patrón esperado dado el desbalance original del dataset (~20% neutras vs ~40% de cada extremo). Los priors uniformes y el `alpha` alto (2.0) fueron las decisiones más determinantes para compensar este desbalance.

Técnicas adicionales exploradas

Como técnica adicional, se exploró el uso de embeddings densos de FastText (`cc.es.300`) combinados con `GaussianNB` (v6). Permitió contrastar dos familias de representación: la dispersa/frecuentista (BoW/TF-IDF con `MultinomialNB`) frente a la densa/semántica (embeddings con `GaussianNB`). El resultado mostró que para esta tarea y con este clasificador, la representación dispersa fue considerablemente superior.

Random Forest

Introducción

Random Forest es un método de ensamble que combina múltiples árboles de decisión entrenados sobre subconjuntos aleatorios del conjunto de datos y de las features. Al promediar las predicciones de muchos árboles distintos, reduce la varianza respecto a un árbol individual y generaliza mejor ante datos no vistos.

Descripción de versiones

v1: RF + TF-IDF, baseline (F1 Kaggle: 0.60657)

Pipeline con TF-IDF por defecto y RandomForest de 100 árboles, `class_weight="balanced"`. Sin búsqueda de hiperparámetros. La clase neutra colapsó a $F1 \approx 0.27$.

v2: RF + TF-IDF + RandomizedSearch 20 iter (F1 Kaggle: 0.65151)

Se incorpora búsqueda aleatoria de hiperparámetros (20 iteraciones, 5-fold CV). Mejores params: `n_estimators=200`, `max_depth=None`, `ngram=(1,2)`, `min_df=3`.

v3: RF + BoW + RandomizedSearch 20 iter (F1 Kaggle: 0.65088)

Se reemplaza TF-IDF por Bag of Words para comparar representaciones. BoW resultó inferior, confirmando que la ponderación IDF es útil para Random Forest.

v4: RF + TF-IDF + RandomizedSearch 40 iter (F1 Kaggle: 0.65170)

Se amplía la búsqueda a 40 iteraciones incorporando `sublinear_tf` al espacio de búsqueda. Descripción completa en la sección siguiente.

v5: RF + TF-IDF + SelectKBest + búsqueda en dos fases (F1 Kaggle: 0.64959)

Pipeline de tres etapas: TF-IDF → SelectKBest(χ^2) → RandomForest. Se incorpora selección de features para reducir ruido, y la búsqueda se realiza en dos fases: una amplia (20 iter, 5-fold) seguida de una refinada alrededor del mejor punto (12 iter, 3-fold). Se exploran dos hiperparámetros nuevos: `min_samples_split` y `max_leaf_nodes`.

v6: RF + TF-IDF + meta-features + dataset LLM balanceado (F1 Kaggle: 0.61096)

Se combinan tres mejoras: dataset aumentado con LLM (~67k filas, distribución 33/33/33), meta-features densas (longitud, puntuación, negaciones, SentiWordNet) concatenadas al TF-IDF, y bosques de hasta 800 árboles. Las features se precomputan una sola vez para evitar recalcularlas durante el CV.

Modelo ganador: v4 - RF + TF-IDF + RandomizedSearch 40 iteraciones

Preprocesamiento

Se aplicó `clean_classical` sobre los textos. Esto incluye conversión a minúsculas, eliminación de puntuación y caracteres especiales y remoción de stopwords en español. Los textos limpios se vectorizan con TF-IDF, que pondera cada término según su frecuencia en el documento y su rareza en el corpus. Esto permite destacar palabras informativas y penalizar términos muy frecuentes.

Representación del texto

Se usó TF-IDF. Los hiperparámetros del vectorizador se incluyeron en la búsqueda:

- `tfidf__ngram_range`: unigramas (1,1) o unigramas+bigramas (1,2)
- `tfidf__min_df`: mínima frecuencia de documento [2, 3, 5]
- `tfidf__sublinear_tf`: [True, False]

Clasificador

RandomForestClassifier, ensamble de árboles de decisión entrenados sobre subconjuntos aleatorios de datos y features. Se usó `class_weight="balanced"` para compensar el desbalance de la clase neutra. Los hiperparámetros buscados fueron:

- `rf__n_estimators`: [100, 200, 300, 400]
- `rf__max_depth`: [None, 50, 80, 100]
- `rf__min_samples_leaf`: [1, 2, 4]
- `rf__max_features`: [sqrt, log2]

Búsqueda de hiperparámetros

Se utilizó `RandomizedSearchCV` explorando 40 combinaciones aleatorias con validación cruzada de 5 folds optimizando F1-macro. Se usó `RandomizedSearchCV` en vez de `GridSearchCV` dado el espacio de hiperparametros de Random Forest.

Mejores hiperparámetros encontrados

Parámetro	Valor
<code>tfidf__ngram_range</code>	(1,2) - unigramas y bigramas
<code>tfidf__min_df</code>	2
<code>tfidf__sublinear_tf</code>	False
<code>rf__n_estimators</code>	300
<code>rf__max_depth</code>	100

<code>rf__min_samples_leaf</code>	2
<code>rf__max_features</code>	sqrt

Entrenamiento final

El mejor estimador encontrado por `RandomizedSearchCV` se reentrenó sobre la unión de train y validación completos antes de generar las predicciones de test, maximizando los datos disponibles para el modelo final.

Resultados

Métrica	Valor
F1 (validación)	0.6418
Precision (validación)	0.6420
Recall (validación)	0.6416
Accuracy (validación)	0.6898
F1 (Kaggle)	0.65170

El modelo mostró buen desempeño en las clases negativa (0.7457) y positiva (0.7712), con menor F1 en la clase neutra (0.4083), patrón esperable dado el menor soporte de esa clase.

Técnicas adicionales exploradas

Como técnicas adicionales se exploraron selección de features con `SelectKBest`(χ^2) sobre la matriz TF-IDF (v5) y la combinación de data augmentation con LLM junto a meta-features densas (v6). La selección de features no mejoró el score de Kaggle respecto a v4 a pesar de obtener mejor F1 local, lo que sugiere que las features eliminadas aportan señal real y no solo ruido.

El experimento más llamativo fue v6: entrenar sobre un dataset balanceado generado por LLM y enriquecer el TF-IDF con meta-features lingüísticas (longitud, signos de puntuación, negaciones, SentiWordNet) elevó el F1 de validación local a 0.7612, pero el modelo cayó a 0.61096 en Kaggle, peor que versiones mucho más simples. Esto indica que el balanceo sintético creó una distribución que no refleja la del test real, y que las meta-features no compensaron ese desfase. Para Random Forest, más datos sintéticos y más features resultaron contraproducentes.

XGBoost

Introducción

XGBoost es un método de ensamble que construye árboles de decisión de forma secuencial, donde cada árbol nuevo corrige los errores del anterior minimizando una función de pérdida de forma iterativa con descenso por gradiente. Incorpora regularización L1 (`reg_alpha`) y L2 (`reg_lambda`) para controlar el sobreajuste, y múltiples hiperparámetros que regulan la complejidad del bosque. Gracias a esto suele superar a métodos como Random Forest en tareas de clasificación con features tabulares o vectorizadas.

Se exploraron ocho versiones progresivas, variando la representación del texto, los hiperparámetros y el dataset utilizado.

Descripción de versiones

v1: XGBoost + TF-IDF, sin búsqueda de hiperparámetros (F1 Kaggle: 0.65274)

Baseline inicial. Pipeline con TF-IDF por defecto y `XGBClassifier(n_estimators=100)`. Se aplicó `sample_weight="balanced"` para compensar el desbalance de clases. Sin ninguna optimización, fue el punto de partida para comparar el efecto de cada mejora.

v2: XGBoost + BoW, sin búsqueda de hiperparámetros (F1 Kaggle: 0.66021)

Mismo setup que v1 pero reemplazando TF-IDF por Bag of Words con configuración por defecto. BoW superó levemente a TF-IDF con la misma configuración, mostrando que la representación sí tiene impacto incluso sin optimización.

v3: XGBoost + TF-IDF + RandomizedSearch 20 iter (F1 Kaggle: 0.66894)

Primera búsqueda de hiperparámetros (`RandomizedSearchCV`, 20 iteraciones, 5 folds, F1-macro). Se exploró: `n_estimators` [100, 200, 300], `max_depth` [3, 5, 7], `learning_rate` [0.05, 0.1, 0.2], `subsample` [0.7, 0.8, 1.0], `colsample_bytree` [0.5, 0.7, 1.0] y `min_child_weight` [1, 3, 5]. La búsqueda sobre TF-IDF permitió subir casi 2 puntos respecto al baseline.

v4: XGBoost + BoW + RandomizedSearch 20 iter (F1 Kaggle: 0.67682)

Misma grilla que v3 pero con BoW. BoW superó a TF-IDF con la misma grilla de búsqueda, consistente con lo observado en v1 vs v2.

v5: XGBoost + BoW + RandomizedSearch 40 iter, grilla refinada (F1 Kaggle: 0.67792)

Se fijaron los mejores valores encontrados en v3/v4 para `max_depth=7`, `learning_rate=0.2`, `colsample_bytree=1.0` y `min_child_weight=1`, y se amplió la búsqueda a 40 iteraciones explorando `n_estimators` [300–700], `subsample` [0.65–0.8]

y regularización L1/L2 (`reg_alpha`, `reg_lambda`). Con BoW y más estimadores se superó v3.

v6: XGBoost + BoW/TF-IDF + dataset aumentado (F1 Kaggle: 0.68157)

Se incorporó data augmentation para balancear las clases. Descripción completa en la sección siguiente.

v7: XGBoost + TF-IDF/BoW + meta-features, sin búsqueda (F1 Kaggle: 0.66826)

Se exploró como técnica adicional el uso de la clase

`VectorizerPlusFeaturesPipeline`, que concatena la representación vectorizada (TF-IDF o BoW) con 12 meta-features numéricas extraídas del texto crudo (longitud, cantidad de palabras, mayúsculas, signos de puntuación, etc.). Sin búsqueda de hiperparámetros el modelo no superó a v5, sugiriendo que las meta-features necesitan ajuste fino para aportar.

v8: XGBoost + TF-IDF/BoW + meta-features + RandomizedSearch (F1 Kaggle: 0.66824)

Combinación de meta-features con búsqueda de hiperparámetros (40 iteraciones). La versión con BoW (v8.2) alcanzó F1 0.66824 en Kaggle, levemente por debajo de v6, confirmando que el data augmentation fue más efectivo que las meta-features para mejorar el desempeño global.

Modelo ganador: v6 - XGBoost + BoW + dataset aumentado

Motivación y dataset

El cuello de botella en las versiones v1–v5 fue el desbalance de la clase neutra (~20% del dataset original). Para resolverlo se entrenó sobre `train_augmented_eda_balanced.csv`: 67.000 filas con distribución ~33%/33%/33%, generado mediante EDA (Easy Data Augmentation con WordNet en español). Se evaluó también un dataset balanceado por LLM (v6.5/v6.6), que obtuvo peores resultados en Kaggle (~0.65 vs ~0.68 con EDA), por lo que el modelo ganador se basa en la augmentation por EDA.

Preprocesamiento

Se aplicó `clean_classical` (minúsculas, eliminación de puntuación y caracteres especiales, remoción de stopwords en español con spaCy `es_core_news_sm`). El texto limpio se vectorizó con `CountVectorizer` (BoW).

Búsqueda de hiperparámetros

El modelo ganador (v6.3) no requirió una búsqueda nueva: se tomaron directamente los mejores parámetros encontrados en v5.2 sobre el dataset original y se aplicaron al dataset aumentado.

Parámetro	Valor
<code>n_estimators</code>	700
<code>max_depth</code>	7
<code>learning_rate</code>	0.2
<code>subsample</code>	0.8
<code>colsample_bytree</code>	1.0
<code>min_child_weight</code>	1
<code>reg_lambda</code> (L2)	2
<code>reg_alpha</code> (L1)	0

Se realizó adicionalmente una búsqueda `RandomizedSearchCV` (40 iteraciones, 5 folds, F1-macro) sobre el dataset aumentado (v6.4), explorando `n_estimators` [400–700], `subsample` [0.65–0.80], `reg_alpha` [0, 0.01, 0.1] y `reg_lambda` [1, 1.5, 2]. Los mejores parámetros hallados fueron `n_estimators`=700, `subsample`=0.75, `reg_lambda`=1, `reg_alpha`=0. Sin embargo, v6.4 obtuvo mejor F1 de validación local (0.7550) pero menor score en Kaggle (0.67492 vs 0.68157 de v6.3), lo que sugiere que la búsqueda sobreajustó a la distribución sintética del dataset aumentado.

Manejo del desbalance

Se usó `compute_sample_weight("balanced", y_train)` para asignar mayor peso a muestras de clases menos frecuentes durante el entrenamiento, tanto en el split de búsqueda como en el reentrenamiento final sobre train+validación completos.

Resultados en validación (v6.3 - BoW, dataset aumentado EDA)

Métrica	Valor
F1-macro (validación)	0.7513
Precision (validación)	0.7516
Recall (validación)	0.7510
Accuracy (validación)	0.7512
F1-macro (Kaggle)	0.68157

Por clase: negativa F1=0.7731, neutra F1=0.6834, positiva F1=0.7973. El balanceo del dataset redujo significativamente la brecha de rendimiento entre clases respecto a las versiones anteriores, donde la neutra llegaba a F1≈0.40.

Técnicas adicionales exploradas

Como técnica no solicitada explícitamente en el TP, se incorporó feature engineering con meta-features numéricas (v7 y v8), implementadas en la clase propia

VectorizerPlusFeaturesPipeline. Las 12 features incluyen longitud del texto, cantidad de palabras, proporción de mayúsculas, cantidad de signos de exclamación/interrogación y otros indicadores estilísticos extraídos del texto crudo. Si bien estas features aportan señal sobre el estilo de escritura del reseñador, los resultados mostraron que no superaron al data augmentation como técnica de mejora: v8 (con meta-features y búsqueda) obtuvo 0.66824 en Kaggle vs 0.68157 de v6.

Red Neuronal

Introducción

El enfoque de red neuronal utilizado en este trabajo se basa en fine-tuning de modelos de lenguaje preentrenados de tipo Transformer (arquitectura BERT/RoBERTa). Estos modelos aprenden representaciones contextuales del texto mediante mecanismos de atención sobre toda la secuencia, capturando dependencias a larga distancia que los modelos BoW y TF-IDF no pueden modelar. El procedimiento general consistió en: tokenización subword del texto con el tokenizer correspondiente al modelo, agregado de una cabeza de clasificación lineal de 3 salidas sobre el token [CLS], y fine-tuning completo con la API `Trainer` de HuggingFace optimizando F1-macro en el conjunto de validación con early stopping. Se aplicó `clean_minimal` como preprocesamiento (solo normalización unicode y colapso de espacios), ya que los tokenizers de transformers esperan texto natural con mayúsculas y puntuación.

Se exploraron trece versiones progresivas, variando el modelo base, el dataset, los hiperparámetros y la estrategia de manejo del desbalance.

Descripción de versiones

v1: BETO + dataset original, configuración estándar (F1 Kaggle: 0.72048)

Baseline con `dccuchile/bert-base-spanish-wwm-cased` (BETO), el modelo BERT entrenado en corpus en español. Dataset original 51k, distribución 40/20/40.

Hiperparámetros: `lr=2e-5`, `epochs=3`, `max_length=128`, `batch_size=32`, `warmup_ratio=0.1`, `weight_decay=0.01`, `fp16`. El F1 de la clase neutra fue de apenas 0.477, el principal cuello de botella del modelo.

v2: BETO + dataset EDA 67k balanceado (F1 Kaggle: 0.70313)

Se reemplazó el dataset original por el aumentado con EDA (67k filas, distribución ~33/33/33). Mismos hiperparámetros que v1. A pesar de que el balanceo del dataset apuntó a mejorar la clase neutra, el score Kaggle fue inferior a v1 (0.70313 vs 0.72048), sugiriendo que el augment EDA no resultó beneficioso para BETO, a diferencia del augment LLM explorado en versiones posteriores.

v3: BETO + dataset LLM aumentado (~66.5k) (F1 Kaggle: 0.72463)

Se usó el dataset aumentado con reseñas sintéticas generadas vía LLM (Groq API), también balanceado. Mismos hiperparámetros que v1. Fue la primera versión en superar F1 0.82 en validación local.

v4: BETO + dataset original, `lr=1e-5`, 5 épocas (F1 Kaggle: 0.72162)

Se exploró un learning rate más bajo (`1e-5`) con más épocas (5), hipótesis de que un ajuste más fino permitiría mejor convergencia. Superó marginalmente a v1 (0.72162 vs 0.72048),

aunque la diferencia es despreciable, indicando que el LR estándar de $2e-5$ ya es casi óptimo para este dataset.

v5: BETO + dataset original, max_length=192 (F1 Kaggle: 0.72132)

Se aumentó el contexto de tokenización a 192 tokens (cubre el percentil 99 de longitud del dataset) para capturar reseñas largas. Superó marginalmente a v1 (0.72132 vs 0.72048), aunque la diferencia es despreciable, sugiriendo que extender el contexto a 192 tokens no aporta información discriminativa relevante adicional para este dataset.

v6: BETO + dataset EDA 67k, lr= $3e-5$, 2 épocas (F1 Kaggle: 0.70365)

Se probó un LR más alto ($3e-5$) con menos épocas sobre el dataset EDA, evaluando la convergencia más rápida. El modelo convergió en 2 épocas con buen resultado en clase neutra ($F1 \approx 0.70$).

v7: BETO + dataset LLM, lr= $1e-5$, 5 épocas (F1 Kaggle: 0.72140)

Combinación del dataset LLM con LR más bajo y más épocas. Resultado comparable a v3, sin mejora significativa respecto al LR estándar.

v8: XLM-RoBERTa-Large + dataset original (F1 Kaggle: 0.72765)

Primera exploración con un modelo distinto a BETO: **xlm-roberta-large** (560M parámetros, Meta AI), un modelo multilingüe preentrenado con mayor capacidad que BETO. Se redujo el batch a 16 y se usaron **gradient_accumulation_steps=2** para un batch efectivo de 32 ante las restricciones de VRAM. Superó a BETO v1 con los mismos datos, validando la elección del modelo para versiones posteriores.

v9: BETO + dataset original, lr= $1e-5$, 5 épocas, post-RoBERTa (F1 Kaggle: 0.72596)

Vuelta a BETO tras confirmar en v8 que XLM-RoBERTa-Large superaba el baseline. Se exploró si BETO con LR más conservador ($1e-5$) y más épocas (5) podía cerrar la brecha con el modelo large. La justificación fue que un LR menor reduce el riesgo de destruir los pesos preentrenados en español, y con early stopping (paciencia=2) el modelo cortaría antes de overfittear. F1-macro local: 0.7184, con neutra en 0.466. No superó a v8, confirmando que la mayor capacidad del modelo XLM-RoBERTa es el factor determinante y no los hiperparámetros de entrenamiento.

v10: XLM-RoBERTa-Base + dataset LLM (F1 Kaggle: 0.71582)

Se probó **xlm-roberta-base** (278M parámetros) con el dataset LLM aumentado y configuración **lr= $2e-5$, epochs=3**. Sin embargo, su score en Kaggle (0.71582) fue inferior a v3 con dataset LLM (0.72463), sugiriendo que **xlm-roberta-base** no aprovecha el dataset aumentado mejor que BETO.

v11: FastText + BiLSTM + MultiHead Attention (F1 Kaggle: 0.66698)

Como técnica adicional se implementó una arquitectura propia no basada en Transformers (descripción completa en sección de técnicas adicionales).

v12: XLM-RoBERTa-Base + split limpio + EDA aug + class weights (F1 Kaggle: 0.72371)

Primera versión con estrategia de split limpio: se realizó el split train/val antes de aumentar, agregando muestras EDA exclusivamente al fold de entrenamiento para evitar leakage sintético en validación. Se incorporaron además `class_weights=[1.0, 2.5, 1.0]` (penaliza 2.5× los errores en clase neutra) y `max_length=256`. El val set contiene 100% muestras reales, dando una estimación más honesta del desempeño.

v13: XLM-RoBERTa-Large + split limpio + EDA aug + class weights (F1 Kaggle: 0.73367)

Descripción completa en la sección siguiente.

Arquitectura del modelo y justificación

Todas las versiones de v1 a v12 (excepto v11) usan la misma arquitectura de fine-tuning de Transformers: un modelo encoder preentrenado con una cabeza de clasificación compuesta por una capa densa + dropout + proyección lineal a 3 clases, aplicada sobre el embedding del token `[CLS]`. La elección de este paradigma se justifica en que los modelos de la familia BERT/RoBERTa generan representaciones contextuales bidireccionales: cada token atiende a todos los demás en la secuencia, capturando relaciones semánticas complejas como negaciones, ironías y ambigüedades que son frecuentes en reseñas de Amazon ("está re bueno mal"). Esto los hace especialmente adecuados para clasificación de sentimientos en texto informal.

La elección de `xlm-roberta-large` sobre BETO se justifica por tres factores:

1. Fue preentrenado con datos en más de 100 idiomas incluyendo español, con mayor corpus que BETO.
2. Usa la arquitectura RoBERTa que elimina la tarea NSP de BERT y entrena con batches más grandes y más datos, obteniendo mejores representaciones.
3. La versión large (560M parámetros vs 110M de BETO) tiene mayor capacidad para distinguir clases ambiguas como la neutra.

Modelo ganador: v13 - XLM-RoBERTa-Large + split limpio + EDA aug + class weights

Modelo base y motivación

`xlm-roberta-large` (Meta AI, 559.893.507 parámetros). Se eligió sobre `xlm-roberta-base` porque en v8 ya se había verificado que la mayor capacidad del modelo large mejoraba el F1 respecto a modelos más chicos con los mismos datos. El entrenamiento se realizó en GPU AMD Radeon RX 9070 XT (17.1 GB VRAM) con ROCm.

Preprocesamiento

`clean_minimal`: normalización unicode (NFKC) y colapso de espacios. Se mantuvo mayúsculas y puntuación para que el tokenizer subword de XLM-RoBERTa (SentencePiece BPE) funcione correctamente.

Dataset y estrategia de split limpio

Se implementó una estrategia de split limpio para evitar leakage de datos sintéticos en validación:

1. Se realizó el split estratificado sobre el dataset original (51k reales) → 40.800 train / 10.200 val.
2. Se extrajeron las 16.000 muestras EDA que no pertenecen al original (12.000 neutras + 2.000 negativas + 2.000 positivas).
3. Solo el fold de train recibió el augment, resultando en 56.800 muestras de entrenamiento con distribución más balanceada (18.320/20.160/18.320).
4. El val set contiene 100% muestras reales, garantizando una estimación sin sesgo.

Hiperparámetros

Parámetro	Valor
<code>learning_rate</code>	1e-5 (conservador para modelo grande)
<code>num_train_epochs</code>	5 (early stopping patience=2)
<code>max_length</code>	256 tokens
<code>per_device_train_batch_size</code>	16 (limitado por VRAM)
<code>gradient_accumulation_steps</code>	2 (batch efectivo = 32)
<code>lr_scheduler_type</code>	cosine
<code>warmup_ratio</code>	0.15
<code>weight_decay</code>	0.01
<code>fp16</code>	True

Manejo del desbalance

Se implementó un `WeightedTrainer` custom que reemplaza la `CrossEntropyLoss` estándar por una versión ponderada por clase: `class_weights=[1.0, 2.5, 1.0]`, penalizando 2.5× los errores en la clase neutra. Esto directamente optimiza la función de pérdida para la clase más difícil, complementando el balanceo del dataset.

Entrenamiento y selección del mejor modelo

Early stopping con paciencia de 2 épocas, evaluando F1-macro en validación al final de cada época. El mejor modelo se alcanzó en la época 4 (F1-macro: 0.7571 en validación local).

Resultados

Métrica	Valor
F1-macro (validación)	0.7571
Precision (validación)	0.7568
Recall (validación)	0.7616
Accuracy (validación)	0.7910
F1-macro (Kaggle)	0.73367

Por clase (validación): negativa F1=0.8371, neutra F1=0.5567, positiva F1=0.8776. La neutra mejoró significativamente respecto a BETO v1 (de 0.477 a 0.557) gracias a la combinación de class weights y EDA aug en el fold de train.

Técnicas adicionales exploradas

Como técnica no solicitada en el TP, en v11 se implementó una arquitectura propia de red neuronal secuencial: FastText + BiLSTM + MultiHead Attention. El modelo tiene ~8.9M parámetros totales (705k entrenables en fase 1) y se entrenó en 2 fases.

Arquitectura

- Capa de embeddings inicializada con vectores FastText preentrenados ([cc.es.300](#), 300 dimensiones, cobertura del vocabulario del 81.4%) con SpatialDropout1D (p=0.2).
- Dos capas BiLSTM apiladas (128 y 64 unidades respectivamente) con LayerNorm entre capas.
- Capa de MultiHead Attention (4 cabezas) con conexión residual y LayerNorm.
- Pooling dual: promedio ponderado por máscara de padding + max pooling, concatenados.
- Capa densa con activación GELU (128 unidades) + Dropout(0.3) → proyección a 3 clases.

Entrenamiento en 2 fases

Fase 1 con embeddings congelados ([lr=1e-4](#), cosine scheduler con reinicios, hasta 30 épocas con early stopping patience=5). Fase 2 desbloqueando embeddings para fine-tuning ([lr=5e-5](#), ReduceLROnPlateau). El preprocesamiento fue [clean_classical](#) (lematización) en lugar de [clean_minimal](#), dado que sin tokenizer subword el vocabulario más limpio reduce el OOV.

Esta versión alcanzó F1-macro 0.7949 en validación, resultado notable para un modelo sin preentrenamiento en texto en español a escala masiva, aunque por debajo de los modelos Transformer completos.

Ensamble

Introducción

El ensamble combina las predicciones de múltiples modelos independientes mediante soft voting ponderado. En lugar de tomar la clase predicha por cada modelo (hard voting), se promedian las probabilidades de cada clase producidas por cada modelo, ponderadas por un peso asignado. La clase con mayor probabilidad promedio es la predicción final. Esto aprovecha la diversidad entre modelos: las redes neuronales basadas en transformers (XLM-RoBERTa, BETO) capturan contexto semántico profundo gracias a mecanismos de atención entrenados sobre grandes corpus de texto, mientras que modelos clásicos como XGBoost y Random Forest explotan patrones frecuentistas sobre representaciones vectorizadas, y Bayes Naive aporta una perspectiva probabilística simple pero efectiva. La combinación tiende a ser más robusta que cualquier modelo individual, ya que cada familia de modelos comete errores distintos y el promedio ponderado los compensan mutuamente.

Descripción de versiones

v1: XLM-RoBERTa-base + XgBoost + Naive Bayes (F1 Kaggle: 0.68914)

Primer ensamble. XLM-RoBERTa-base se entrena directamente en el notebook con `lr=2e-5, epochs=3, max_length=128`. XGBoost y Naive Bayes se cargan desde archivos joblib previamente entrenados. Los pesos se asignaron proporcionalmente al F1-macro de Kaggle de cada modelo individual, resultando en valores casi iguales (~0.351 / ~0.329 / ~0.320). Se detectó un error de implementación: ambas rutas sklearn apuntaban al mismo archivo (`nb_bow_gridsearch.joblib`), de modo que el ensamble combinó efectivamente RoBERTa + Naive Bayes + Naive Bayes en lugar de incluir XGBoost. A pesar de esto, el ensamble mejoró respecto a Naive Bayes individual.

v2: BETO v3 + BETO v9 + RoBERTa-BNE n1 (F1 Kaggle: 0.73555)

Ensamble de tres transformers sin modelos clásicos. Se combinan dos versiones de BETO: una entrenada sobre dataset aumentado con LLM (v3) y otra con `lr=1e-5, epochs=5` (v9), junto a RoBERTa-BNE n1 entrenado sobre el dataset original. La motivación fue explorar si combinar transformers de distinta configuración mejora respecto a uno solo. Pesos: 0.25 / 0.35 / 0.40. El ensamble (F1 local: 0.7476) resultó inferior al mejor modelo individual (BETO v3: 0.7744), evidenciando que agregar modelos más débiles sin suficiente diversidad de familias diluye el resultado.

v3: RoBERTa-BNE n1 + XGBoost v6+ Random Forest v4 (F1 Kaggle: 0.73930)

Primera combinación de un transformer con dos modelos clásicos de familias distintas. RoBERTa-BNE n1 aporta representación semántica profunda sobre texto en español, XGBoost v6 fue el mejor modelo clásico individual y Random Forest v4 con TF-IDF agrega una tercera perspectiva frecuentista. El primer run usó pesos 0.40/0.35/0.25 sobre el dataset original (F1 local: 0.7972, F1 Kaggle: 0.73853). Se probaron luego múltiples sub-versiones con distintas combinaciones de pesos y datasets (original, EDA augmented y

LLM augmented), donde las versiones con dataset augmentado mostraron F1 locales inflados (hasta 0.8567). La mejor configuración (pesos 0.40/0.30/0.30 sobre dataset original) alcanzó el techo de 0.73930 en Kaggle. Descripción completa en la sección siguiente.

v4: RoBERTa-BNE n1 + XgBoost + Naive Bayes con dataset EDA (F1 Kaggle: 0.73345)

Se reemplaza Random Forest por Bayes Naive y la validación se realiza sobre el dataset aumentado con EDA (~67k filas, distribución 33/33/33). El mayor peso a XGBoost refleja su mejor desempeño individual sobre datos balanceados. Pesos: 0.30 / 0.50 / 0.20.

v5: RoBERTa-BNE n1 + XGBoost + NB con dataset LLM (F1 Kaggle: 0.73345)

Igual configuración que v4 pero la validación se realiza sobre el dataset aumentado con LLM, con mayor peso al transformer (0.50 / 0.30 / 0.20). F1 local de 0.8509, el más alto del proyecto, aunque sujeto al mismo problema de comparabilidad que v4: el conjunto de validación contiene datos sintéticos del mismo LLM, lo que infla la métrica local.

v6: XLM-RoBERTa-base LLM + RF v5 + XGBoost (F1 Kaggle: 0.72157)

Se prueba un transformer entrenado específicamente sobre el dataset balanceado por LLM ($lr=2e-5$, $epochs=3$), junto a RF v5 y XGBoost. Se le da mayor peso a XGBoost por su mejor desempeño individual. Pesos: 0.25 / 0.25 / 0.50.

v7: BETO v3.1 + XGBoost v6 (F1 Kaggle: 0.72828)

Se redujo el ensamble a dos componentes reemplazando RoBERTa-BNE n1 por BETO v3.1 y eliminando Random Forest. XGBoost v6 se reentrenó directamente en el notebook sobre el dataset EDA augmented con los hiperparámetros de v6 (BoW , $n_estimators=700$, $max_depth=7$, $lr=0.2$). Los pesos se asignaron manualmente priorizando el transformer por encima de lo que sugería el F1 individual (BETO: 0.7744, XGBoost: 0.7862). Pesos: 0.60 / 0.40.

v8: XLM-RoBERTa N13 Large + RoBERTa-BNE n1 + XGBoost + RF + Naive Bayes (F1 Kaggle: 0.72695)

Se amplió el ensamble a cinco modelos sumando el transformer de mayor capacidad entrenado en el trabajo (XLM-RoBERTa N13 Large, con split limpio, EDA aug y class weights) al ya utilizado en los ensambles anteriores (RoBERTa-BNE n1). Los modelos clásicos (XGBoost v6.3, RF v4 y Naive Bayes) se cargaron desde joblib. La evaluación se realizó sobre el dataset LLM augmented (~66.5k filas). Se exploraron tres distribuciones de pesos dando mayor protagonismo a los transformers: 0.40/0.30/0.10/0.10/0.10 (F1 Kaggle: 0.72478), 0.31/0.30/0.13/0.13/0.13 (F1 Kaggle: 0.72695) y 0.26/0.26/0.16/0.16/0.16 (F1 Kaggle: 0.71859). La mejor configuración fue la segunda.

v9: XLM-RoBERTa-Large N13 + XGBoost + RF + Complement NB + Logistic Regression (F1 Kaggle: 0.73746)

Se amplió el ensamble a cinco modelos para incorporar mayor diversidad. Como transformer se usó XLM-RoBERTa N13 Large, ya entrenado con split limpio y class weights. A su vez se usaron XGBoost con EDA aug, RF v5 sin augmentation y, como técnica adicional, Complement Naive Bayes y Logistic Regression entrenados con LLM aug. Además, se usó la misma estrategia de split limpio que en las redes neuronales v12/v13. En lugar de asignar los pesos manualmente, se los optimizó con Nelder-Mead minimizando el F1-macro negativo sobre el val limpio; el resultado fue que el optimizador concentró casi todo el peso en XLM-RoBERTa (0.45), XGBoost (0.29) y RF (0.22), asignando pesos cercanos a cero a CNB (0.02) y LR (0.02), lo que sugiere que esos dos modelos no aportaron diversidad útil sobre el transformer. Pesos finales: 0.45 / 0.29 / 0.22 / 0.02 / 0.02.

Modelo ganador: v3 - RoBERTa-BNE n1 + XGBoost v6 + RF v4

Motivación

El v3 fue seleccionado como modelo definitivo para la tabla. Combina tres modelos de familias distintas: un transformer preentrenado en español (RoBERTa-BNE n1) que captura contexto semántico profundo, XGBoost que demostró el mejor desempeño individual entre los modelos clásicos (F1: 0.8385), y Random Forest como tercer componente con representación TF-IDF. La diversidad entre los modelos es clave: cada uno comete errores distintos, y el promedio ponderado los compensa.

Modelos ensamblados

Modelo	Peso
RoBERTa-BNE n1	0.4
XGBoost v6	0.3
Random Forest v4	0.3

Pesos

Los pesos se asignaron manualmente priorizando el modelo con mayor F1 individual (RoBERTa-BNE n1 con 0.40) y distribuyendo el resto equitativamente entre XGBoost y RF.

Preprocesamiento

Se aplicaron dos pipelines según el tipo de modelo. Para el transformer se usó `clean_minimal` (preserva mayúsculas y acentos, solo limpieza mínima). Para los modelos sklearn se usó `clean_classical` (minúsculas, eliminación de puntuación, remoción de stopwords con spaCy `es_core_news_sm`). Cada modelo recibe el texto en el formato con el que fue entrenado originalmente.

Mecanismo de ensamble

Para cada modelo se obtiene una matriz de probabilidades (n, 3), una columna por clase, usando softmax sobre los logits del transformer y `predict_proba` para los modelos sklearn. Se calcula el promedio ponderado de estas matrices y la clase con mayor probabilidad resultante es la predicción final.

Resultados

Métrica	Valor
F1 macro (validación)	0.7932
Precision (validación)	0.7957
Recall (validación)	0.7913
Accuracy (validación)	0.8326
F1 macro (Kaggle)	0.73930

Por clase: negativa F1=0.8737, neutra F1=0.6054, positiva F1=0.9005. El ensamble mejoró respecto a cada modelo individual (RoBERTa-BNE n1: 0.7327, XGBoost: 0.8385, RF: 0.7652).

Técnicas adicionales exploradas

Se exploraron ensambles con datasets de validación aumentados (EDA y LLM) en v4 y v5, que mostraron F1 de validación considerablemente más altos (0.8370 y 0.8509 respectivamente). Sin embargo, estos valores no son comparables directamente con el resto de la tabla ya que la validación se realizó sobre conjuntos distintos al split estándar. También se probó un ensamble de solo transformers (v2) y de solo dos modelos (v7), confirmando que la combinación de modelos de familias diversas es la estrategia más efectiva.

Conclusiones generales

El análisis exploratorio del dataset resultó fundamental para orientar las decisiones del trabajo. Identificar que la clase neutra representaba solo el 20% del total, frente al 40% de cada clase extrema, explicó de antemano por qué todos los modelos mostraron dificultades para clasificar correctamente esa categoría, y motivó las estrategias de data augmentation, class weights y split limpio implementadas en etapas posteriores. También fue el EDA quien reveló la naturaleza informal del texto y la ausencia de ruido estructural (sin HTML, URLs ni menciones), lo que habilitó una limpieza minimalista en lugar de un pipeline agresivo que podría haber eliminado señales útiles.

El preprocesamiento tuvo un impacto diferencial según el tipo de modelo. Para los modelos clásicos (Bayes Naive, Random Forest, XGBoost), la limpieza con `clean_classical` (lematización, eliminación de stopwords) redujo el espacio de features y mejoró la señal disponible, siendo clave para superar los baselines. Sin embargo, para los modelos transformer el preprocesamiento relevante fue el opuesto: `clean_minimal` preserva mayúsculas, puntuación y acentos que los tokenizers subword necesitan para funcionar correctamente. El efecto más pronunciado del preprocesamiento fue sobre la clase neutra: ningún pipeline logró resolver completamente su ambigüedad semántica, ya que reseñas como "llegó bien pero la calidad es regular" requieren comprensión contextual que los modelos frecuentistas no capturan.

En cuanto al desempeño en TEST, el mejor resultado en Kaggle fue el ensamble v3 (RoBERTa-BNE n1 + XGBoost v6+ Random Forest v4) con F1-macro 0.73930. El F1 local más alto del proyecto fue el ensamble v5 (RoBERTa-BNE n1 + XGBoost + NB con dataset LLM), con 0.8509 en validación, aunque no es directamente comparable con el resto: v5 fue evaluado sobre el split del dataset aumentado con LLM (~66.5k filas, distribución balanceada 33/33/33), mientras que los demás ensambles se evaluaron sobre el split del dataset original.

El modelo más sencillo y rápido de entrenar fue Bayes Naive v5 (MultinomialNB + BoW + GridSearch), que se entrena en segundos con CPU y alcanza un F1-macro de 0.66461 en Kaggle. En relación a su simplicidad, este resultado es sorprendentemente competitivo: logra más del 90% del desempeño del mejor modelo individual utilizando una fracción ínfima del tiempo y recursos computacionales. Para tareas donde la latencia o el costo son críticos, constituye un baseline sólido y difícil de ignorar.

Sobre la viabilidad productiva del mejor modelo, el ensamble v3 presenta desafíos importantes. Requiere cargar en memoria tres modelos de tamaños muy distintos (el transformer solo ocupa varios GB), la latencia de inferencia es alta comparada con modelos clásicos, y el mantenimiento de un pipeline de tres modelos es considerablemente más complejo. Para un caso de uso real, dependería del trade-off entre precisión y restricciones operativas: si el volumen de predicciones es bajo y la calidad es prioritaria, el ensamble es viable; si se necesita alta disponibilidad y bajo costo, XGBoost v6 (F1 Kaggle: 0.68157) sería preferible por su velocidad de inferencia y tamaño reducido.

En cuanto a posibles mejoras que quedaron fuera del alcance del trabajo, se destacan varias líneas. El fine-tuning de XLM-RoBERTa-Large sobre el dataset aumentado con LLM (combinando mayor capacidad del modelo con mayor cantidad de datos balanceados) no fue explorado por restricciones de tiempo y cómputo, y podría haber sido el experimento más prometedor. También quedó pendiente aplicar la estrategia de split limpio con el dataset LLM aumentado, ya que solo se usó con EDA en v12 y v13. Finalmente, técnicas como label smoothing, mixup de embeddings, o búsqueda más exhaustiva de pesos del ensamble mediante validación cruzada podrían haber incrementado el F1-macro del modelo combinado.

Una dimensión que no fue explorada en profundidad es la diversidad lingüística del dataset. Si bien las reseñas están nominalmente en español, es probable que una fracción de ellas esté escrita en otros idiomas (principalmente inglés o portugués), ya sea en su totalidad o mezclada con el español en forma de code-switching. Esta presencia de textos multilingües podría haber motivado decisiones de modelado distintas: por ejemplo, filtrar o identificar esos registros antes del entrenamiento, o priorizar modelos multilingües como XLM-RoBERTa que fueron preentrenados sobre más de 100 idiomas y podrían manejar mejor esa diversidad. Un análisis exploratorio más detallado sobre los idiomas presentes en el corpus habría sido una herramienta útil para orientar estas decisiones.

Una pregunta que también nos planteamos fue si el preprocesamiento tuvo un efecto diferencial sobre el desbalance entre clases, y la respuesta fue que sí, de forma asimétrica. El pipeline `clean_classical` mejoró el desempeño general de los modelos clásicos al reducir el ruido léxico, pero su impacto fue muy desigual entre clases: negativa y positiva se beneficiaron más porque tienen vocabulario más marcado (palabras fuertes de valoración), mientras que la clase neutra, cuyas reseñas son inherentemente ambiguas y equilibradas, resultó más afectada por la eliminación de stopwords y la lematización, que borran matices como intensificadores y conectores concesivos ("aunque", "pero", "sin embargo") que son precisamente los que diferencian una reseña neutra de una polarizada. En los modelos transformer, el uso de `clean_minimal` preservó esos elementos y permitió que la atención contextual los aprovechara, lo que se tradujo en una mejora consistente del F1 de la clase neutra respecto a los modelos clásicos, aunque sin llegar a resolver completamente el problema dado el desbalance estructural del dataset original.

Respecto a la representación del texto, los embeddings contextuales de los modelos transformer resultaron la opción más efectiva por un margen amplio. XLM-RoBERTa-Large alcanzó 0.73367 en Kaggle frente a 0.68157 de XGBoost con BoW y 0.66461 de Bayes Naive con BoW. Entre las representaciones clásicas, BoW superó a TF-IDF en la mayoría de los experimentos para XGBoost y Bayes Naive, probablemente porque la normalización IDF penaliza términos frecuentes que en este dominio (reseñas de consumo) son efectivamente informativos. FastText con GaussianNB fue la representación con peor resultado ($F1 \approx 0.50$), ya que el promedio de embeddings por documento pierde información posicional y las distribuciones gaussianas por feature no modelan bien representaciones densas de alta dimensión.

Otro interrogante que surgió durante el desarrollo fue la brecha sistemática entre el score de validación local y el obtenido en Kaggle. En todos los modelos se observó esta diferencia,

siendo el caso más llamativo el ensamble v5, que alcanzó 0.8509 en validación pero solo 0.73345 en Kaggle. Esto se explica principalmente por dos factores: el uso de datasets aumentados sintéticamente como conjunto de validación, que no refleja fielmente la distribución real del test, y el sobreajuste implícito al split local tras múltiples iteraciones de búsqueda de hiperparámetros. El ensamble v3, evaluado sobre el split del dataset original, mostró la brecha más honesta y consistente (0.7932 vs 0.73930).

Una pregunta que también nos planteamos fue por qué la clase neutra resultó tan difícil de clasificar en comparación con las otras dos. En el mejor modelo individual (XLM-RoBERTa-Large v13), la neutra obtuvo $F1=0.5567$ frente a 0.8371 de la negativa y 0.8776 de la positiva. La explicación combina tres factores: su subrepresentación en el dataset original (~20%), la ambigüedad semántica intrínseca de sus reseñas (que mezclan valoraciones positivas y negativas en el mismo texto) y el hecho de que su vocabulario comparte términos con ambos extremos, lo que reduce su separabilidad en cualquier espacio de representación.

Finalmente, nos preguntamos si el data augmentation resultaría beneficioso de manera uniforme para todos los modelos, y la respuesta fue que no. Para XGBoost, el dataset aumentado con EDA mejoró el F1 de Kaggle de 0.67792 a 0.68157. Para los modelos transformer, en cambio, el EDA no resultó beneficioso, mientras que el dataset generado por LLM sí aportó una mejora marginal. Para Random Forest, el augment con LLM fue directamente perjudicial (0.61096), sugiriendo que la distribución sintética creó un desfase respecto al test real. En conclusión, el augment es sensible tanto al modelo consumidor como al método de generación, y no puede asumirse beneficioso de forma genérica.

Tiempo dedicado

Integrante	Tarea	Promedio hs semanales
Bermudez, Agustin	Armado de reporte Armado de notebooks Entrenamiento de modelos	8
Calderón, Tiago	Armado de reporte Armado de notebooks Entrenamiento de modelos	8
Gonzalez Pautaso, Mateo	Armado de reporte Armado de notebooks Entrenamiento de modelos Templates	10
Moreyra, Santiago	Armado de reporte Armado de notebooks Entrenamiento de modelos	8
Nieves, Maylen	Armado de reporte Armado de notebooks Entrenamiento de modelos	8